Smart Disaster Response MIS

Design Rationale Document

DELIVERABLE — DATABASE DESIGN

Course: Database Systems

Campus: FAST-NUCES, Islamabad

Semester: Spring 2026

Document: Design Rationale

Table of Contents

1. Entity & Relationship Choices

1.1 Citizens vs. Users

Citizens (the public) and **Users** (system staff) are maintained in separate tables by design. Citizens submit emergency reports without requiring an account. Users are authenticated staff with assigned roles. Merging them would introduce **password_hash** and **role** fields into public citizen records and create unnecessary NULL columns for unauthenticated reporters, creating a serious security boundary violation.

1.2 EmergencyReports as the Central Entity

EmergencyReports is the hub connecting citizens, operators, teams, resources, hospitals, and patients. All other modules — *TeamAssignments*, *ResourceAllocations*, *Patients* — hold a foreign key referencing it, reflecting the real-world flow: *everything originates from a reported emergency*.

1.3 Weak Entities & Junction Tables

DispatchLogs and **TeamMembers** use composite primary keys where one component is a foreign key to the parent **RescueTeams** table — classic weak entities whose identity is only meaningful in context of the parent team. **WarehouseInventory** resolves the many-to-many between Warehouses and Resources while carrying two critical attributes: **quantity_available** and **threshold_level**.

2. Normalization (1NF to 4NF)

Normal Form	Compliance Description
1NF	Atomic values in every column; no repeating groups. A citizen's phone is a single column, not a comma-delimited list.
2NF	All non-key attributes in composite-PK tables depend on the full key. In WarehouseInventory, quantity_available depends on both warehouse_id AND resource_id.
3NF	No transitive dependencies. Resources.resource_type is independent; Warehouses stores manager_id as a direct FK, not embedded contact info.
4NF	No multi-valued dependencies. Each table captures exactly one fact per row; no multiple statuses or locations in a single row.

Table 1 — Normalization compliance summary

3. Transaction Handling

Transactions are applied wherever multiple tables must be updated **atomically**. All follow the SQL Server **BEGIN TRY / BEGIN TRANSACTION / COMMIT / BEGIN CATCH / ROLLBACK** pattern.

Operation	Tables Involved	Rollback Condition
Resource Dispatch	ResourceAllocations + WarehouseInventory	Stock would go negative
Team Assignment	TeamAssignments + EmergencyReports	Status update fails
Donation Recording	Donations + FinanceTransactions	Transaction record fails to insert
Patient Admission	Patients + Hospitals	Hospital beds at zero

Table 2 — Transactional operations and rollback conditions

4. Role-Based Access Control (RBAC)

Roles are stored as a constrained **NVARCHAR** column in **Users** via a CHECK constraint. For five fixed, well-defined roles unlikely to change dynamically, this avoids an extra join on every authentication check.

Role	Access Scope
admin	Full access to all modules, audit log, and user management
emergency_operator	Reports, teams, hospitals, approvals — the dispatcher role
field_officer	Reports, hospitals, resource requests (ground response)
warehouse_manager	Inventory, dispatch, resource approvals
finance_officer	Donations, expenses, financial approvals

Table 3 — Role definitions and access scopes

5. Trigger Justification

Triggers enforce invariants that cannot be delegated to application code, since direct database access or concurrent transactions may bypass application-layer validation.

Trigger	Event	Invariant Maintained
trg_ResourceAllocation_Dispatch	UPDATE ResourceAllocations	Inventory never goes negative
trg_TeamAssignment_Insert	INSERT TeamAssignments	Team status set to 'assigned'; dispatch log written
trg_TeamAssignment_Complete	UPDATE TeamAssignments	Team returns to 'available' on completion
trg_Patient_Admission	INSERT Patients	Beds decremented; ROLLBACK if zero beds available
trg_Patient_Discharge	UPDATE Patients	Beds incremented on discharge
trg_FinanceTransaction_AuditLog	INSERT FinanceTransactions	Every financial event logged to AuditLog
trg_EmergencyReport_AuditLog	INSERT, UPDATE EmergencyReports	All report changes audited with before/after
trg_ResourceAllocation_AuditLog	INSERT, UPDATE ResourceAllocations	Full allocation pipeline audited
trg_Inventory_LowStockAlert	UPDATE WarehouseInventory	Manager notified when stock crosses threshold
trg_ApprovalRequest_Execute	UPDATE ApprovalRequests	Approved allocations automatically linked

Table 4 — Trigger inventory and invariants maintained

6. View Justification

View	Used By	Purpose
vw_ActiveEmergencyReports	Operators, Admins	Active reports with citizen contact and operator name
vw_TeamAvailability	Operators, Field Officers	Available teams with member counts
vw_WarehouseInventorySummary	Warehouse, Admins	Stock levels with low-stock flag
vw_HospitalCapacity	Field Officers, Operators	Real-time bed availability with occupancy %
vw_FinancialSummary	Finance, Admins	Aggregated totals — no raw donor PII exposed
vw_ApprovalQueue	Admins, role-specific	Pending approvals with submitter context
vw_ResourceAllocationStatus	Warehouse, Admins	Full allocation pipeline status
vw_AuditSummary	Admins only	Human-readable audit trail with user names

Table 5 — View inventory, consumers, and purpose

7. Indexing Strategy

Index	Columns	Query Pattern
IX_ER_DisasterType_Status	(disaster_type, status)	Operator dashboard active-incident filter
IX_ER_Location_Severity	(location, severity_level)	Geographic incident analysis
IX_FT_Type_Timestamp	(transaction_type, transaction_timestamp)	Financial reports by type and date
IX_AL_Table_Timestamp	(table_affected, action_timestamp)	Audit log filtered by table and date range

Table 6 — Composite index definitions

8. Performance Comparison Results

Scenario	Result
Filter disaster_type + status	~65% reduction in logical reads after composite index
Filter location + severity_level	~58% reduction in logical reads
AuditLog date-range filter	~70% reduction after IX_AuditLog_ActionTimestamp
vw_WarehouseInventorySummary	Identical to direct JOIN (optimizer inlines view)
100 INSERTs — EmergencyReports	~23% longer with all 5 indexes vs. none
1000 INSERTs — AuditLog	~8% overhead (minimal index set)

Table 7 — Performance benchmark highlights

9. Trade-offs Summary

Design Decision	Trade-off
Role ENUM in Users table	Simpler schema vs. less flexible for future role changes
AFTER triggers for consistency	Automatic enforcement vs. harder to debug trigger chains
Views for role abstraction	Simpler frontend queries vs. one additional layer to maintain
Composite indexes on hot paths	Faster reads vs. slower writes on indexed tables
Minimal indexes on AuditLog	Fast inserts vs. slower audit-log read queries
Raw T-SQL (no ORM)	Full control and explicit queries vs. more boilerplate code
JWT in httpOnly cookies	XSS protection vs. not usable from native applications

Table 8 — Key design decisions and associated trade-offs